

Doomsday Algorithm — Final Event Plan

Data Science Club · VIT-AP · V-TAPP 2026

Live site	https://doomsdayalgorithm-5f1e8.web.app SMARTPANTS PRESERVED 55SMARTPANTS PRESERVED 56SMARTPANTS PRESERVED 57
Date	September 11, 2026
Window	9:00 AM – 6:00 PM (9 hours)
Format	Drop-in. Teams arrive, play, and leave whenever. No group kickoff.
Runtime per team	~2.5–3 hours across four rounds
Last comfortable start	~3:00 PM
Cost	₹0 — no credit card anywhere in this setup
Status	Deployed and verified end-to-end. Ready to run.

1. The premise

Teams are handed a dataset from **Earth-4471** — a parallel Marvel universe. Real character names, entirely fabricated facts. Four completed “Phases” of film history (28 films, 60 characters) are public. **Phase 5 has not happened**, and teams must predict what it holds.

This design exists for one reason: the answers cannot be looked up. Not on Google, not from an LLM, not from knowing the real MCU. Every fact in this universe was generated for this event. A banner on every page reminds them: *“This is not Earth-616. Anything you remember about the real MCU is wrong here.”*

The dataset contains **three deliberate statistical traps**. Teams that model naively will find strong, confident, wrong answers:

- **Simpson’s paradox** — pooled across all phases, screentime correlates *negatively* with box office. Within any single phase, it correlates *positively*. Teams that don’t segment get the sign backwards.
- **Survivorship bias** — `roster.csv` has 60 characters, `appearances.csv` has fewer. The missing ones aren’t dead; they haven’t debuted yet. Teams that infer from appearances alone systematically misjudge who’s available for Phase 5.
- **Leaky column** — `final_billing_position` is a strong survival predictor in historical data, but it only exists *because those films already released*. Phase 5 rows structurally cannot have it. A model built on it can’t be applied at all.

Catching a trap is worth points in Round 4.

2. What a team actually does

The landing page is a **step-by-step round tracker** — each round shown in order with its own instructions, time estimate, and entry button. There is no live host, so the site carries the entire briefing.

ROUND 0 — SET UP · 5 MINUTES

Pick a team ID (saved in the browser — same laptop throughout). Download six CSVs:

File	Contents
<code>films.csv</code>	28 films: phase, year, budget, opening weekend, worldwide gross, critic & audience scores
<code>characters.csv</code>	60 characters: faction, first film, powered flag
<code>appearances.csv</code>	Per character per film: screentime, dialogue lines, billing order, final billing position, survived
<code>co_appearances.csv</code>	Character pairs sharing scenes, per film
<code>post_credits.csv</code>	Who was teased, and which film paid it off
<code>roster.csv</code>	The full 60-character roster

Excel, pandas, SQL, R — whatever they have.

ROUND 1 — EXPLORE · 30–45 MINUTES · 10 QUESTIONS

Every answer is already in the files they just downloaded. Nothing to predict. This round exists to force an actual read of the data before Round 2 asks them to extrapolate from it.

Question types: film-to-film gross comparisons, “did this character survive every film they appeared in”, “which of these four has the most total screentime across 28 films”, faction lookups, post-credits payoff lookups. Cross-file joins and per-character aggregates, not single-cell lookups.

Answered as probabilities, same as every round. Graded on submission.

ROUND 2 — PREDICT · 60–90 MINUTES · 24 QUESTIONS

About Phase 5, which hasn’t happened. No file contains these answers.

Type	Count	Example
Survival	10	"Will Thanos survive Phase 5?"
Team-up	6	"Will Loki appear in a team-up (2+ shared scenes) in Phase 5?"
Closest partner	4	"Which character does Shuri share the most scenes with in Phase 5?" (multiple choice)
Screentime comparison	4	"Will Vision have more total screentime than Nebula in Phase 5?"

Each answer is a **probability from 0–100%**, not a guess. Scored with a Brier score — a proper scoring rule, meaning the strategy that maximises your expected score is to state what you actually believe. Bluffing 95% and being wrong costs more than honestly saying 60%.

ROUND 3 — DRAFT · 5 MINUTES

Claim **one character** believed to survive Phase 5. **Exclusive** — first team to claim a character takes them off the board for everyone else, enforced atomically in the database (not just in the page’s UI, so two browser tabs can’t cheat it).

The pool holds **43 characters** — every character with a real Phase 5 outcome, drawn from the 60-character roster. If more than 43 teams register, the last ones find the pool empty; that’s inherent to an exclusive draft over a fixed roster.

This is the only round with genuine time pressure. Draft scores post for everyone at once at the end, after the organizer reveals Phase 5 — it can’t be scored per-team before then, since it depends on data nobody sees until the reveal.

ROUND 4 — REPORT · 20–30 MINUTES

The page auto-fills their boldest Predict answers. For each, they write 1–2 sentences citing the specific chart or number that backs it, and name at least one trap they caught and how they handled it. A judge scores it by checklist.

3. Scoring

$$\text{Total} = 0.5 \times \text{Analysis\%} + 0.2 \times \text{Draft\%} + 0.3 \times \text{Report\%}$$

Analysis% (50%) — Explore and Predict share one bucket. Sum of all 34 Brier scores ÷ 34.

For each question: $\text{points} = 100 \times (1 - \text{penalty})$, where penalty is $(1-p)^2$ if the answer turns out yes, or p^2 if no.

You said	Truth	Points
100%	Yes	100
90%	Yes	99
50%	Either	75
90%	No	19
100%	No	0

A skipped question defaults to the neutral **75** — an honest shrug is never worse than a guess. (Caveat: this applies once a team has at least one submission. A team that never submits anything at all scores 0, because there's no record to credit.)

Draft% (20%) — $(\text{survived} +30, \text{top-third screentime} +20, \text{had a team-up} +15) \div 65$, capped at 100.

Report% (30%) — judge's checklist below, averaged across however many judges score it.

Tie-break — higher raw Analysis score, then earlier submission timestamp.

The projector (`admin.html`) shows **rank and total only**, never a per-round breakdown. This is deliberate: a visible raw Predict score would let a team submit 100% on a throwaway team ID, read the score back, and extract the answer key one question at a time.

4. Judging — Round 4 checklist

One sheet per team. **Tick boxes, do not free-score.** Team IDs only, no names — grade blind. Budget 60–90 seconds per report. Each category caps at 25; total 100.

1 · DATA SOPHISTICATION — 25 PTS

- ☐ Combined more than one data table (not just one CSV in isolation) — **+10**
- ☐ Used a real statistic — rate, trend, or correlation — not just a raw number — **+10**
- ☐ The tool/method shown actually matches the claim being made — **+5**

2 · LOGIC & EVIDENCE — 25 PTS

- ☐ Prediction #1 cites a specific chart/number — **+5**
- ☐ Prediction #2 cites a specific chart/number — **+5**
- ☐ Prediction #3 cites a specific chart/number — **+5**
- ☐ Correctly identified and handled a planted trap (Simpson's paradox / survivorship bias / leaky column) — **+10**

3 · VISUALIZATION — 25 PTS

- ☐ At least one chart is included — **+10**

- ☐ Chart is readable — axes labeled, clear what it shows — **+10**
- ☐ Chart actually supports the claim placed beside it (not decorative) — **+5**

4 · COMMUNICATION — 25 PTS

- ☐ Fits the half-page limit, no rambling — **+5**
- ☐ A stranger could follow the reasoning in under a minute — **+10**
- ☐ States confidence level AND reasoning together, not just a claim — **+10**

Enter the total into the portal's judge queue. Do not hand-total across multiple judges — the system averages them.

Printable version: `Doomsday_Algorithm_Judging_Checklist.docx` in the repo.

5. Organizer run sheet

BEFORE 9:00 AM

1. Open a terminal and start the grading server — **leave this window open all day**:

```
cd portal/local-server
node grading-server.js
```

It must print `Loaded 34 questions ... Answer key: present`. If it says **MISSING**, re-run `seed.js` (see `RUNBOOK.md`) — grading silently does nothing without the answer key.

2. Put `admin.html` on the projector for live standings.
3. Have team IDs ready to assign at check-in (see limitations).

9:00 AM – 6:00 PM

Nothing. Keep the grading server window open and let people drop in. Judging can start as soon as reports appear — no need to wait for the end.

AT 6:00 PM (OR ONCE THE DRAFT POOL HAS HAD A FAIR RUN)

4. In a **second** terminal, leaving the grading server running:

```
cd portal/local-server
node reveal.js
```

This scores every team's draft pick and posts the final combined leaderboard. Draft scores appear on the projector within seconds. This is the one synchronized moment of the whole event.

5. Announce winners from the projector.

If teams are still working at 6 PM: either let them finish before running the reveal, or run it and let their picks score alongside everyone else's.

6. How it runs for free

- **Firebase Firestore, Spark (free) plan** — database, security rules, and static hosting. No credit card, ever.

- **No Cloud Functions.** Firebase requires the paid Blaze plan for Cloud Functions even to stay inside the free quota. Instead, `portal/local-server/grading-server.js` runs on the organizer's own laptop for the whole window, doing exactly what a Cloud Function would: grading submissions and judge scores the instant they land. Writes are debounced and diffed, so even a burst of simultaneous submissions never approaches Firestore's free daily limits.
- `seed.js` loads questions and the answer key into Firestore before the event (already done for this deployment).
- `reveal.js` is a one-shot script for the end of the day.
- The answer key lives in Firestore under rules that deny **all** client access. Only the laptop scripts, authenticated with a service account key, can read it.

Deployment details: project `doomsdayalgorithm-5f1e8`, owned by nj58711@gmail.com — the Firebase CLI must be logged in as *that* account. Dataset seed `42`. Full setup steps in `RUNBOOK.md`.

7. Known limitations — accepted, not bugs

- **Team IDs are self-chosen, not authenticated.** A team could type another team's ID. Mitigate by assigning IDs at check-in.
 - **No automated window enforcement.** Firestore doesn't check the clock; a team could technically submit outside 9–6. Compensate operationally: treat anything after `reveal.js` runs as not counting.
 - **Draft pool caps at 43 characters**, not team count. Teams arriving very late may find it thin or empty. Expected, not broken.
 - **A very late arrival won't finish all four rounds.** With a 9-hour window this only affects teams starting after ~3 PM. Explore, Predict and Report are each graded per submission, so partial progress keeps every point it earned, and the landing page tells late teams to start anyway.
 - **A team with zero submissions scores 0%**, not the neutral 75% a partially-completing team gets — there's no record to credit for a team that never appears in the data at all.
 - **The grading server must stay running.** If the laptop sleeps or the terminal closes, submissions still save but aren't graded until it restarts (it catches up automatically on restart).
-

8. Verification status

Verified end-to-end against the live deployment on 2026-09-10 — not locally, not in an emulator:

- ☒ Landing page, all four round pages load with zero console errors
- ☒ Explore serves exactly 10 questions; Predict serves exactly 24
- ☒ A real submission graded through to `leaderboard` and `leaderboard_public` within seconds
- ☒ Draft pool serving all 43 characters
- ☒ Report page correctly loading a team's own prior submissions
- ☒ Raw per-round scores confirmed unreadable by clients; only rank + total public
- ☒ Dataset generator: 27/27 tests passing, all three traps verified present in shipped data
- ☒ Test data deleted — leaderboard is empty and ready

Nothing further is required before the event beyond starting the grading server.